Architecture Design

for

Mentor Caring System

Version 1 approved

Prepared by Ma Xinyao(FUll), Guan Xinling(FUll), Hou Shuoran(FUll), Peng Yancheng(FUll), Tang Jianxu(FUll), Ouyang Hao(FUll)

B09

2026.04.16

Contents

Revision History

Name	Date	Reason For Changes	Version
B09	2026.04.16	Initial approved version	1.0

1. Overview

1.1 Project description

The Mentor Caring System is a web-based supporting system for the BNBU Mentor Caring Programme. It supports mentor allocation, interview records, appointments, communication, summary report generation, and information tracking.

1.2 References

Design and Implementation of a Mentor Caring System, Project Long Description, Version 1.2, 2025–2026 Semester 2.

1.3 Design purpose

This document describes the architecture of the Mentor Caring System. It is used to support system design, implementation, and future maintenance.

2. Overall description

2.1 Use case diagram and class diagram

The use case diagram shows the interactions between the system and its main user roles, including Administrator, Faculty Consultant, Mentor, Student, MCP Coordinator, and Supporting Staff.

The class diagram shows the main entities in the system and their relationships.

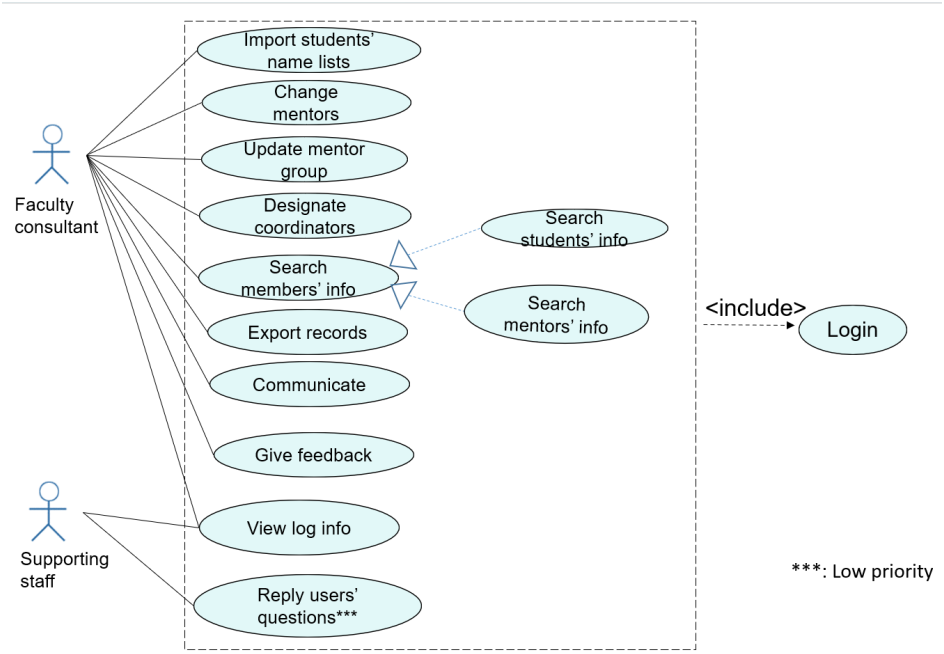

Figure 1

Figure 1: Use Case Diagram for Faculty Consultant and Supporting Staff

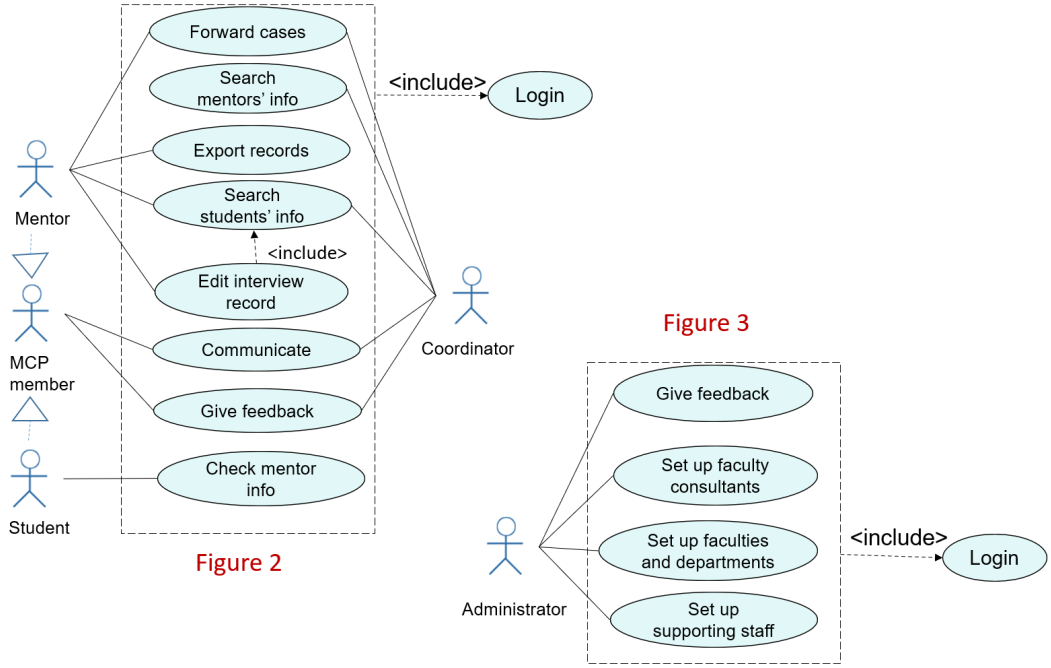

Figure 2: Use Case Diagram for Mentor, MCP Member, Student, Coordinator, and Administrator

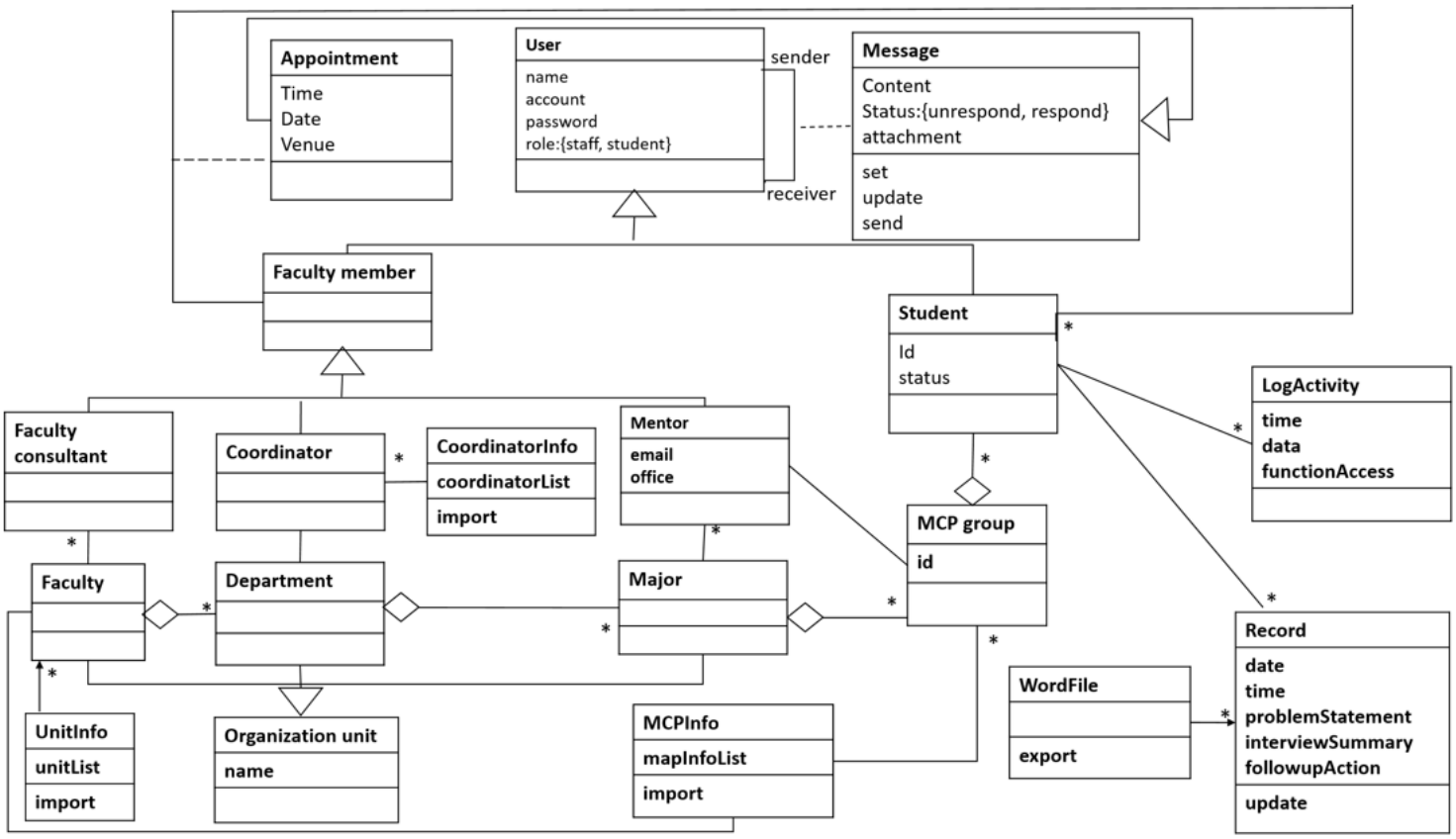

Figure 3: Class Diagram of the Mentor Caring System

2.2 Design model

The system uses a layered web system architecture. It can be divided into presentation layer, business logic layer, data access layer, and database layer. This model is chosen because it separates responsibilities clearly and supports easier maintenance and extension. It is also suitable for a role-based university information system with multiple functions and users.

2.3 System architecture

Based on the object-oriented design, the overall architecture of the Mentor Caring System is constructed by grouping related domain classes into interacting subsystems, as shown in Figure 4.

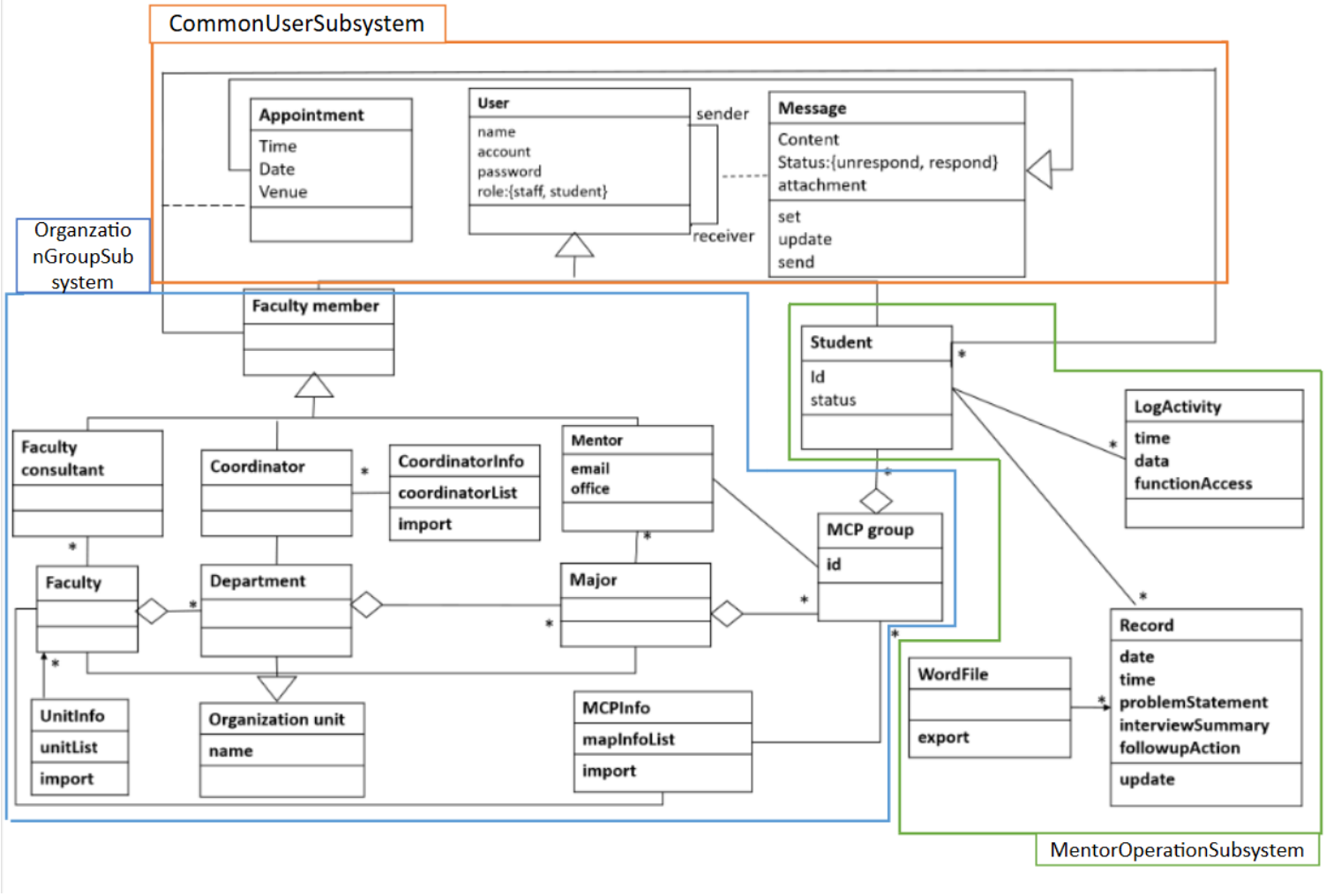

Figure 4: Subsystem grouping of the Mentor Caring System

According to Figure 4, the system is mainly divided into the following subsystems.

1. **Common User Subsystem**
   This subsystem contains the shared classes used by multiple roles in the system. Its main classes include `User`, `Message`, and `Appointment`. These classes support common functions such as authentication, role-based access, communication, and interview appointment management.
2. **Organization and Group Allocation Subsystem**
   This subsystem contains the structural and allocation-related classes of the system. Its main classes include `Faculty`, `Department`, `Major`, `Faculty consultant`, `Coordinator`, `Mentor`, `MCP group`, `Organization unit`, `UnitInfo`, `CoordinatorInfo`, and `MCPInfo`. These classes are used to maintain the university structure, staff assignments, and mentoring-group allocation.
3. **Mentoring Operation Subsystem**
   This subsystem contains the classes related to the core mentoring workflow. Its main classes include `Student`, `Record`, `LogActivity`, and `WordFile`. These classes support student-related operations, interview record maintenance, activity logging, and document export.

These subsystems are associated with each other and together support the complete workflow of the Mentor Caring System. The detailed internal architecture and database design of these specific subsystems are elaborated in Sections 3.1, 3.2, and 3.3, respectively.

Besides the internal subsystem grouping, the Mentor Caring System also interacts

with several external resources in the university environment, as shown in Figure 5. The system connects with the **School User Database** for authentication and user identity information. It also uses **Excel Import Files** to initialize and update organization structure and allocation data. In addition, the system interacts with the **University Email Service** for message notification and supports **Word File Export** for report and record output.

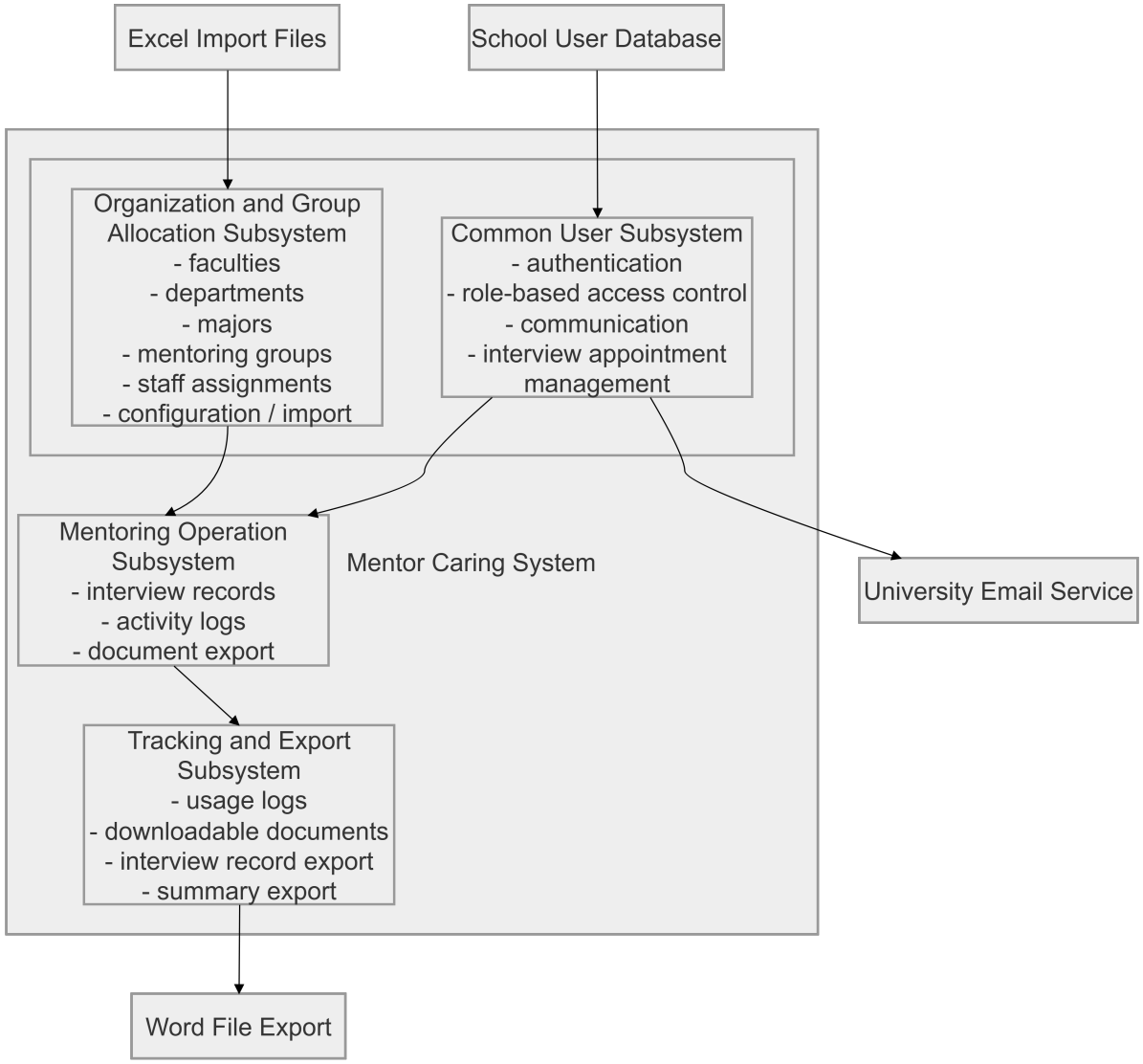

Figure 5: Overall context of the Mentor Caring System

3. System architecture

This section describes the main subsystems in the Mentor Caring System, including their responsibilities, internal components, and related database design.

3.1 Common User Subsystem

The Common User Subsystem is responsible for the shared functions used by different roles in the Mentor Caring System. It mainly handles authentication, role-based access control, communication, and interview appointment management. This subsystem is divided into four layers: UI, Controls, Persistency, and Database layer.

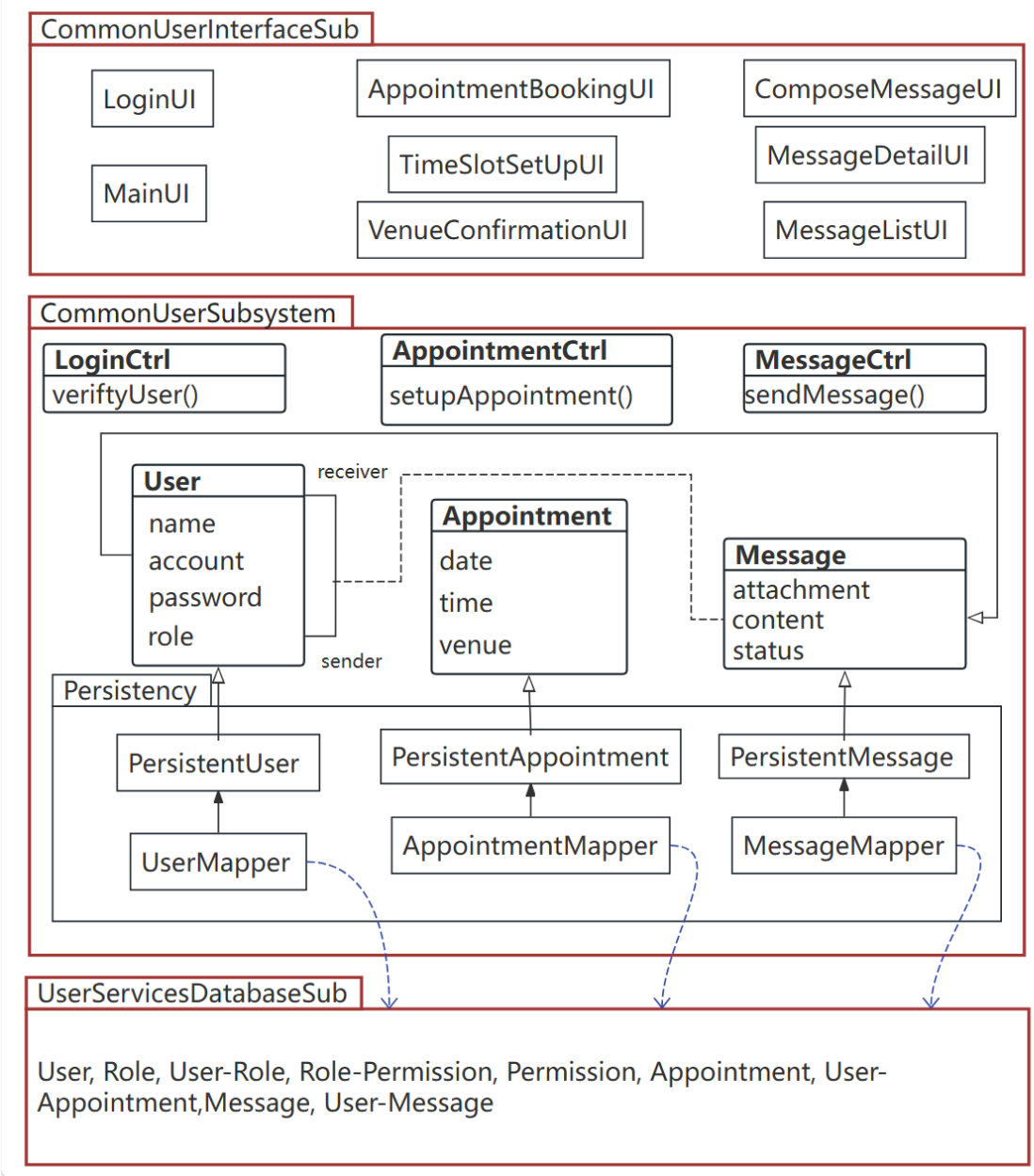

Figure 6: Architecture diagram of the Common User Subsystem

3.1.1 Description

LoginUI & MainUI: Responsible for capturing user credentials for authentication and serving as the central navigation dashboard based on the user's role.

AppointmentBookingUI, TimeSlotSetUpUI, VenueConfirmationUI: Boundary objects that handle the interactions for the interview arrangement process, allowing mentors to set availability and confirm venues, and students to book time slots.

ComposeMessageUI, MessageDetailUI, MessageListUI: Interfaces that allow users to view their inbox, read specific message content, and send or broadcast communications to other users.

LoginCtrl: Manages the authentication workflow, verifies user credentials (`verifyUser()`), and enforces role-based access before routing to the appropriate interface.

AppointmentCtrl: Executes the core business logic for interview scheduling (`setupAppointment()`), including validating available time slots and updating appointment statuses.

MessageCtrl: Handles the communication logic (`sendMessage()`), processes single

or multiple receivers, and updates message reading statuses.

User: A domain object representing the core attributes of all system participants, such as students, mentors, and staff.

Appointment: Encapsulates the business rules, time, date, venue, and state of an interview arrangement.

Message: Represents the content, type, and attachment data of communications passed between users.

PersistentUser, PersistentAppointment, PersistentMessage: Subclasses of the domain entities that contain database-related states needed for persistence.

UserMapper, AppointmentMapper, MessageMapper: Data Access Objects that connect the object-oriented logic and the relational database.

UserInfoDatabaseSub, AppointmentDatabaseSub, MessageDatabaseSub: Logical storage modules that contain the physical database tables for authentication, communication, and appointment management.

3.1.2 Database

User				
Account	Password	Name	Email	UserId

Message			
MessageId	Content	Status	Attachment

User-Message		
SenderId	MessageId	ReceiverId

Appointment					
AppointmentId	Time	Date	Venue	Status	MessageId

User-Appointment		
InitiatorId	AttendeeId	AppointmentId

Role			
RoleId	RoleName	RoleCode	RoleDescription

Permission		
PermissionId	PermissionName	PermissionCode

User-Role		
Id	UserId	RoleId

Role-Permission		
Id	RoleId	PermissionId

Figure 7: Main database tables of the Common User Subsystem

The database design of this subsystem is divided into three parts.

1. **UserInfoDatabaseSub (User Authentication and Authorization)**
   User Table: Stores the core credentials and access level of all system participants.
   Role Table: Defines the specific user roles within the system for access control.
   Permission Table: Defines the specific privileges for system operations.
   User_Role Table: A mapping table linking the many-to-many relationship between users and roles.
   Role_Permission Table: A mapping table linking roles to their authorized permissions.
2. **MessageDatabaseSub (Communication Management)**
   Message Table: Stores the core content of communications sent by users.
   User_Message Table: A mapping table that handles message broadcasting and tracks individual delivery status.

3. **AppointmentDatabaseSub (Interview Scheduling)**
   Appointment Table: Records the scheduling details of interview arrangements.
   User_Appointment Table: Stores the relationship between appointment initiators, attendees, and appointments.

3.2 Organization and Group Allocation Subsystem

The Organization and Group Allocation Subsystem is responsible for managing the university structure, staff role allocation, and MCP group allocation in the Mentor Caring System. It maintains faculties, departments, majors, mentoring groups, and related configuration data, and provides basic allocation data for other subsystems.

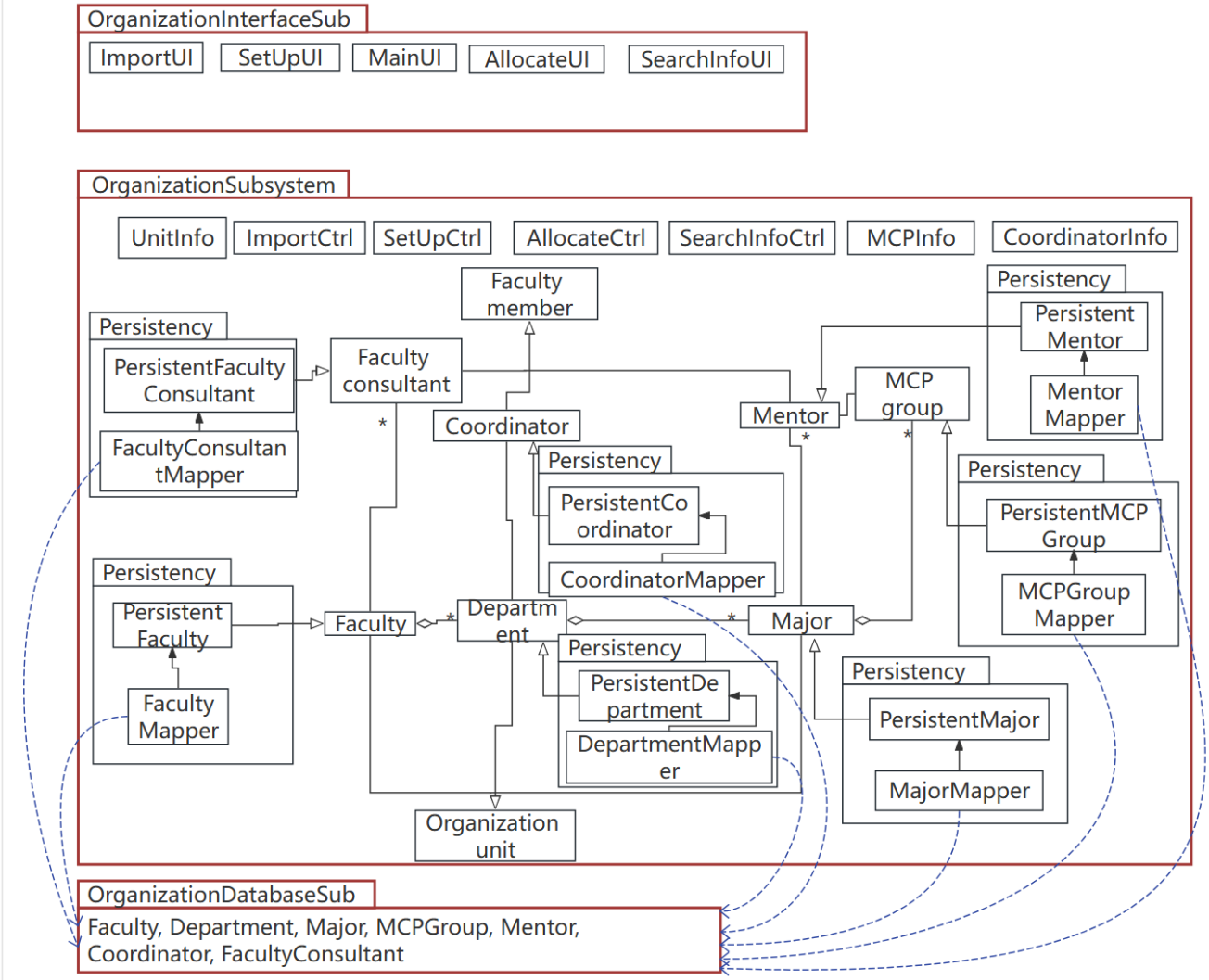

Figure 8: Architecture diagram of the Organization and Group Allocation Subsystem

3.2.1 Description

The Organization and Group Allocation Subsystem contains the components used to maintain the university structure, staff role allocation, MCP group allocation, and related configuration data. It supports the setup and maintenance of faculties, departments, majors, mentoring groups, and staff assignments.

Organizational Structure Components The organizational structure components include Faculty, Department, Major, and Organization Unit. These components represent the academic hierarchy of the university and maintain the basic structure of faculties,

departments, and majors. They provide the foundation for role allocation and mentoring group arrangement in the system.

Staff Role Components These components include Faculty Member, Faculty Consultant, Coordinator, and Mentor. They represent the main staff roles in the mentoring programme and support role assignment in different academic units. In particular, mentors are related to the formation and management of MCP groups.

Configuration / Import Components These components include UnitInfo, CoordinatorInfo, and MCPInfo. They are used to import and update organization-related and group-allocation data, which improves the efficiency of system initialization and maintenance. MCPInfo is specifically used to support the mapping information related to MCP groups.

3.2.2 Database

The persistent data in this subsystem is stored in several database tables related to organization structure, staff role allocation, and MCP group allocation. The main tables are shown below.

Faculty	
facultyID	facultyName

Department		
departmentID	departmentName	facultyID (for association)

Major		
Major	majorName	departmentID (for association)

MCPGroup		
groupID	majorID (for association)	mentorID (for association)

FacultyConsultant		
consultantID	consultantName	facultyID (for association)

Coordinator			
coordinatorID	coordinatorName	email	departmentID (for association)

Mentor				
mentorID	mentorName	email	office	majorID (for association)

3.3 Mentoring Operation Subsystem

This subsystem handles interview records, activity logs, and document export. It has three layers: MentoringInterfaceSub (UI), MentoringSub (entities, controllers, and Persistency sub-packages holding mappers and persistent classes), and MentoringDatabaseSub (physical database tables).

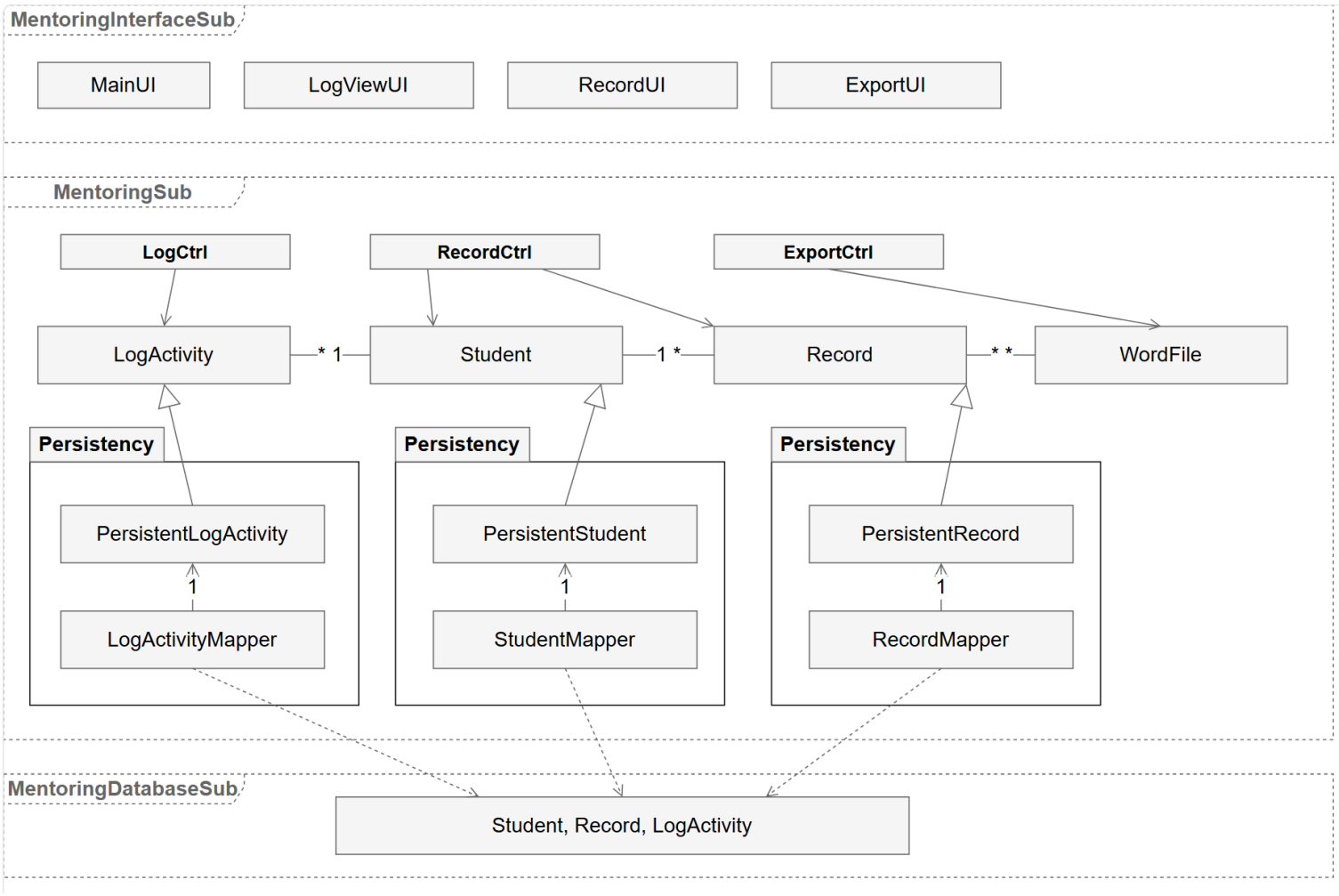

Figure 9: Architecture diagram of the Mentoring Operation Subsystem

3.3.1 Description

MainUI, RecordUI, LogViewUI, ExportUI — screens for navigation, record editing, log viewing, and Word export.

Student, Record, LogActivity, WordFile — entity classes. Attributes follow the class diagram; Student inherits `name`, `account`, `password`, and `role` from `User`. `WordFile` is a generated artifact and is not persisted.

RecordCtrl, LogCtrl, ExportCtrl — controllers for record CRUD, activity logging, and export. `RecordCtrl` uses `Student` to locate the owner of each record and read profile fields needed for export.

StudentMapper, RecordMapper, LogActivityMapper — translate between entity objects and their database tables.

PersistentStudent, PersistentRecord, PersistentLogActivity — persistent subclasses of the corresponding entities, stored in the database.

3.3.2 Database

Student `name`, `account`, `password`, and `role` come from the school database and are not duplicated here. Faculty and department are resolved through Student → Major → Department → Faculty. `Record.Mentor_ID` is stored explicitly because mentors change every academic year but records must remain attributable.

Table 1: Student Table

Student_ID	Major (for association)	Status	Group_ID (for assocation)

Table 2: Record Table

Record_ID	Date	Time	Problem_ Statement	Interview_ Summary	Followup_ Action	Student_ID (for association)	Mentor_ID (for association)	Group_ID (for association)

Table 3: LogActivity Table

Log_ID	User_ID (for association)	Time	Function_Access	Data

4. Assessment

4.1 Stability

4.2 Reusability

4.3 Scalability

5. Alternative design (optional)

6. More considerations

7. Appendix